

Year Up United Data Analyst Academy

Pre-Work Week 1 Student Guide

This courseware is to be distributed to accompany instructor led training.

Table of Contents

MODULE 1 FILE SYSTEMS & THE COMMAND LINE	3
SECTION 1.A TECHNICAL FLUENCY FOUNDATIONS	4
1.A.1 Why Analysts Need Technical Fluency.....	5
SECTION 1.B UNDERSTANDING FILE SYSTEMS.....	6
1.B.1 Understanding File Systems.....	7
1.B.2 Absolute vs Relative Paths	8
1.B.3 Why Folder Structure Matters for Analysts.....	9
SECTION 1.C INTRODUCTION TO THE COMMAND LINE WITH GIT BASH	10
1.C.1 Introduction to Git Bash.....	11
1.C.2 Why Use the Command Line	12
1.C.3 The Git Bash Prompt.....	13
1.C.4 The Working Directory.....	14
SECTION 1.D ESSENTIAL CLI COMMANDS.....	15
1.D.1 pwd.....	16
1.D.2 ls.....	17
1.D.3 cd.....	18
1.D.4 mkdir	19
1.D.5 touch	20
1.D.6 rm.....	21
1.D.7 rmdir.....	22
1.D.8 Command Mindset.....	23
 MODULE 2 INTRODUCTION TO VERSION CONTROL WITH GIT	 24
SECTION 2.A WHY VERSION CONTROL MATTERS	25
2.A.1 The Problem Version Control Solves	26
2.A.2 What is Version Control?	27
SECTION 2.B CORE GIT CONCEPTS.....	28
2.B.1 Repository.....	29
2.B.2 Commit.....	30
2.B.3 Staging.....	31
2.B.4 Remote	32
SECTION 2.C THE BASIC GIT WORKFLOW	33
2.C.1 The Git Workflow at a Glance	34
2.C.2 Initializing a Local Repository.....	35
2.C.3 Creating a Remote Repository	36
2.C.4 Connecting Local to Remote	37
2.C.5 Adding and Committing Changes.....	38
2.C.6 Pushing to the Remote Repository	39
2.C.7 Authenticating with GitHub.....	40
2.C.8 Verifying Your Work	41
2.C.9 Cloning a Repository.....	42
2.C.10 Git Beyond the Basics.....	43

Module 1

File Systems & The Command Line

Section 1.A

Technical Fluency Foundations

1.A.1 Why Analysts Need Technical Fluency

- Before we begin working with Excel, SQL, or Python, we need to talk about something more basic:
- *How you interact with your computer*
- Professional analysts are not just “good at data”
- They are:
 - ◇ Organized
 - ◇ Precise
 - ◇ Comfortable in technical environments
 - ◇ Able to troubleshoot independently
- If your structure is chaotic, your work becomes chaotic
- You can easily:
 - ◇ Overwrite important work
 - ◇ Lose track of files
 - ◇ Break a project workflow
- Technical fluency is not about being *advanced*
- It is about being *reliable*

Section 1.B

Understanding File Systems

1.B.1 Understanding File Systems

- Your computer stores information using a hierarchical structure
- You can think of it like a tree
- At the top is a root directory
 - ◇ Below that, there are folders (also called directories)
 - * Inside folders are more folders and files
- In Windows, your root typically starts with a drive letter
- Example:

```
C:\Users\yourname\Documents\DataAnalytics\
```

- This structure might look like:

```
C:\
├── Users
│   ├── yourname
│   │   ├── Documents
│   │   │   ├── DataAnalytics
│   │   │   │   ├── raw_data
│   │   │   │   ├── cleaned_data
│   │   │   │   └── reports
```

- Each level builds on the one above it
- This is called a directory hierarchy

1.B.2 Absolute vs Relative Paths

- Every file on your computer has a “**path**” – a precise address to where that file is located
- An **absolute path** starts from the root drive
- Example:

```
C:\Users\yourname\Documents\DataAnalytics\raw_data
```

- A **relative path** starts from your current working location
- If you are already inside the `DataAnalytics` folder, the relative path is just:

```
raw_data
```

- Understanding paths is critical for:
 - ◇ Running scripts
 - ◇ Connecting files
 - ◇ Working with programming tools
 - ◇ Avoiding “file not found” errors

1.B.3 Why Folder Structure Matters for Analysts

- Good organization is a foundational professional skill
- A **clean folder** structure:
 - ◇ Prevents accidental overwrites
 - ◇ Makes work reproducible
 - ◇ Makes collaboration easier
 - ◇ Signals professionalism
- A **messy folder** structure:
 - ◇ Slows you down
 - ◇ Confuses teammates
 - ◇ Increases risk of errors
 - ◇ Makes debugging painful
- Disorganized projects become harder to manage as they grow
- Organization isn't just for aesthetics – it is vital for managing your work
- Clear structure makes it easier for others to review and validate your work

Section 1.C

Introduction to the Command Line with Git Bash

1.C.1 Introduction to Git Bash



- Throughout this course, you will use **Git Bash** as your command line tool
- A **command line tool** lets you interact with your computer by typing instructions instead of clicking buttons
 - ◇ Instead of clicking folders, you type commands
 - ◇ Instead of dragging files, you instruct your computer precisely what to do
- Git Bash allows you to use **text-based commands** to:
 - ◇ Navigate folders
 - ◇ Create files
 - ◇ Remove files
 - ◇ Run programs
- Many professional Data Analyst tools operate through command line interfaces
- Learning to use them builds technical confidence

1.C.2 Why Use the Command Line

- You might be wondering:
- *If I can click folders, why would I type commands?*
- The command line:
 - ◇ Enables efficient file management
 - ◇ Powers Git and other technical tools we'll soon introduce
 - ◇ Encourages precision
 - ◇ Supports automation
- Even if you don't become a heavy command line user, you must be comfortable with it
- Analysts regularly use:
 - ◇ Git
 - ◇ Python
 - ◇ Package managers
 - ◇ Cloud tools
- All of these rely on the command line

1.C.3 The Git Bash Prompt

- When you open Git Bash, you will see something like:



◇ This is called the prompt

- It tells you:

- ◇ **Who** you are logged in as
- ◇ **What** environment you are in
- ◇ **Where** you are currently located

- The  symbol represents your **home directory**

- ◇ On Windows, that typically maps to:

```
C:\Users\yourname
```

- Git Bash displays Windows path slightly differently

```
/c/Users/yourname
```

- This still represents your Windows **C:\Users\yourname** directory, just displayed in a different format

1.C.4 The Working Directory

- The terminal always has a **current working directory**
- Every command you type runs from that location
- If you are in the wrong directory, your command may:
 - ◇ Fail
 - ◇ Affect the wrong files
 - ◇ Create confusion
- This is one of the most common mistakes for *beginners* and *experienced users* alike
- Before running commands, be sure to confirm where you are
- You can do this using the `pwd` command

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~  
$ pwd  
/c/Users/timmk
```

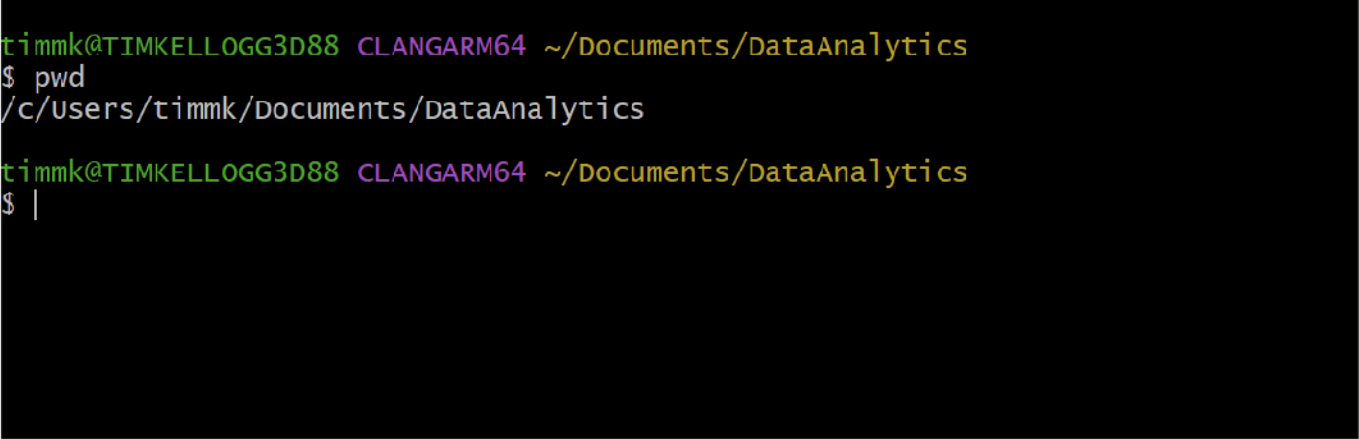
- This command returns your **current working directory** location

Section 1.D

Essential CLI Commands

1.D.1 pwd

- Prints **w**orking **d**irectory
- Shows your current location in the file system

A terminal window with a black background and yellow text. The prompt is 'timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics'. The user enters 'pwd' and the output is '/c/users/timmk/Documents/DataAnalytics'. The prompt is then followed by a vertical bar '|'.

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics
$ pwd
/c/users/timmk/Documents/DataAnalytics

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics
$ |
```

- Helps you confirm your **c**urrent **w**orking **d**irectory location

1.D.2 ls

- **List**
- Shows the files and folders inside your current directory

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics
$ ls
pre-week-1/  pre-week-2/  pre-week-3/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics
$
```

◇ If you don't see what you expect, you may be in the wrong location!

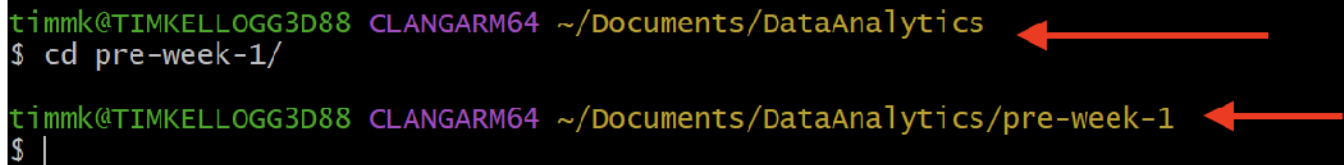
- Input a directory name after the **ls** command to list the contents of another folder without navigating to it

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics
$ ls pre-week-1
old-folder/
```

1.D.3 cd

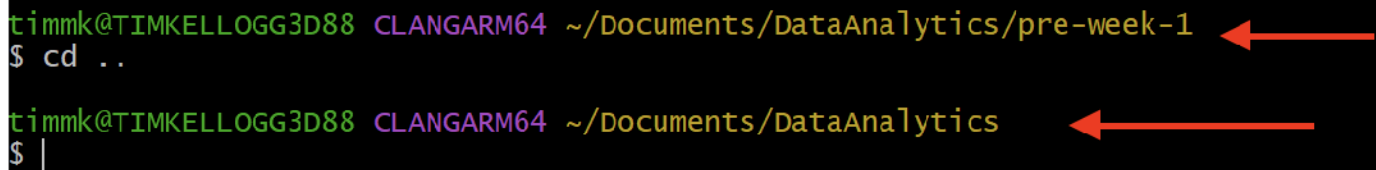
- **Change directory**
- Moves you into a different folder

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics  
$ cd pre-week-1/  
  
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1  
$ |
```



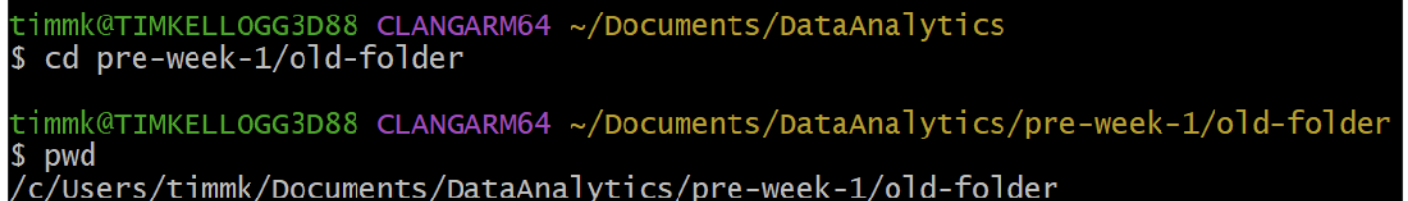
- Use `cd ..` to move up one level

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1  
$ cd ..  
  
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics  
$ |
```



- `cd` can also navigate through multiple directories at once using relative paths

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics  
$ cd pre-week-1/old-folder  
  
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1/old-folder  
$ pwd  
/c/Users/timmk/Documents/DataAnalytics/pre-week-1/old-folder
```



1.D.4 mkdir

- **Make directory**

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ mkdir new-folder

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
new-folder/  old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ |
```

- Use `ls` command to verify new folder has been created

1.D.5 touch

- Creates a new empty file

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
new-folder/  old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ touch notes.txt

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
new-folder/  notes.txt  old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ |
```

- Use `ls` command to verify new file has been created

1.D.6 rm

- Removes a file

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
new-folder/  notes.txt  old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ rm notes.txt

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
new-folder/  old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$
```

- Be careful of typos!
 - ◇ The command line assumes you meant what you typed

1.D.7 rmdir

- Removes a **directory**

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
new-folder/  old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ rmdir new-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls
old-folder/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ |
```

- By default, won't work on directories with files in them

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ ls old-folder/
some-file.txt

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ rmdir old-folder/
rmdir: failed to remove 'old-folder/': Directory not empty

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics/pre-week-1
$ |
```

◇ This is a guardrail to prevent accidental file deletions

1.D.8 Command Mindset

- Computers:
 - ◇ Do exactly what you tell them
 - ◇ Mean exactly what they say
 - ◇ Do not infer your intent
- The command line is precise
- If something breaks, it is usually because:
 - ◇ You are in the wrong directory
 - ◇ A file name is misspelled
 - ◇ A space or character is missing
 - ◇ The path is incorrect
- Most errors are small
- Precision prevents confusion
- Errors are not *failure*, they are *feedback*
- Learning to troubleshoot is one of the most valuable technical skills you can build

Module 2

Introduction to Version Control with Git

Section 2.A

Why Version Control Matters

2.A.1 The Problem Version Control Solves

- Imagine this scenario:
- You are working on a project. You save your file as:

```
analysis.xlsx
```

- You make updates

```
analysis_final.xlsx  
analysis_final_v2.xlsx  
analysis_final_REAL.xlsx
```

- Now imagine two people are editing the same file
 - ◇ Who made which change?
 - ◇ Which version is correct?
 - ◇ What was modified?
 - ◇ How do you undo changes?
- Without version control:
 - ◇ Files get overwritten
 - ◇ Changes are lost
 - ◇ Collaboration becomes confusing
 - ◇ Accountability is limited
- Version control exists to solve this exact problem

2.A.2 What is Version Control

- Version control is a system that tracks changes to files over time
- It allows you to:
 - ◇ See **what** changed
 - ◇ See **when** it changed
 - ◇ See **who** changed it
 - ◇ **Restore** earlier versions
 - ◇ **Collaborate** without overwriting work
- **Git** is the most widely used version control system
- It tracks change efficiently and reliably

Section 2.B

Core Git Concepts

2.B.1 Repository

- A repository (repo) is a project folder that Git tracks
- A repository contains:
 - ◇ Files
 - ◇ Folders
 - ◇ A complete history of changes
- Every time you commit, Git records a new snapshot inside the repository
- This allows you to:
 - ◇ See what changed
 - ◇ Restore earlier versions
 - ◇ Trace the evolution of a product
- A repository turns a simple folder into a version-controlled project

2.B.2 Commit

- A commit is a snapshot of your project at a specific moment
- Each commit records:
 - ◇ What changed
 - ◇ When it changed
 - ◇ A message describing the change
- Commits create a timeline of your work
- This timeline allows you to:
 - ◇ Review earlier versions
 - ◇ Compare changes between commits
 - ◇ Restore a previous state if something breaks
- A commit is not just ‘saving a file’
- It is intentionally recording a meaningful change
- Clear commit messages make your thinking visible
- Over time, your **commit history** tells the story of your project

2.B.3 Staging

- Staging is the step between making changes and committing them
- When you modify files, Git notices changes
- Staging allows you to decide which of those changes will be included in the next commit
- This gives you control over what is recorded
- You might:
 - ◇ Stage one file but not another
 - ◇ Stage only part of your work
 - ◇ Organize related changes into a single commit
- Staging prevents accidental commits
- It encourages thoughtful, intentional snapshots
- Without staging, commits would be less precise

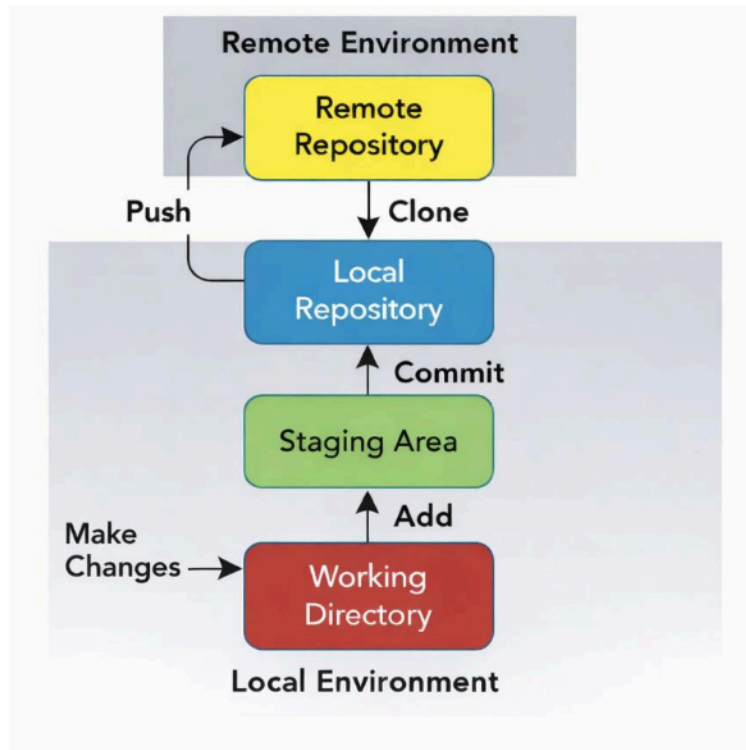
2.B.4 Remote

- A remote repository is a version of your project stored online
- It exists outside your local computer
- Remote repos allow:
 - ◇ Backup
 - ◇ Sharing
 - ◇ Collaboration
 - ◇ Recovery if your device fails
- When you push commits to a remote repository:
 - ◇ Your local history is copied to the remote
 - ◇ Others can see your changes (if you want them to)
 - ◇ Your work is preserved beyond your machine
- GitHub is a popular platform for hosting remote Git repositories
- **Git** is the tool that tracks the change
- **GitHub** stores and displays those tracked changes online
- *Remote repositories protect your project from local device failure*

Section 2.C

The Basic Git Workflow

2.C.1 The Git Workflow at a Glance



- At a high level, Git follows this cycle:
 1. Initialize a local repository
 2. Connect it to a remote repository
 3. Make changes
 4. Stage changes
 5. Commit changes
 6. Push to remote
- Each step has a purpose
- Understanding the full cycle makes the commands easier to place

2.C.2 Initializing a Local Repository

- Before Git can track changes, you must initialize a repository
- For this example, create a new folder outside of your DataAnalytics folder (in Documents) called `git-test`

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents
$ mkdir git-test

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents
$ cd git-test/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test
$
```

- Initialize your Git repository using the `git init` command

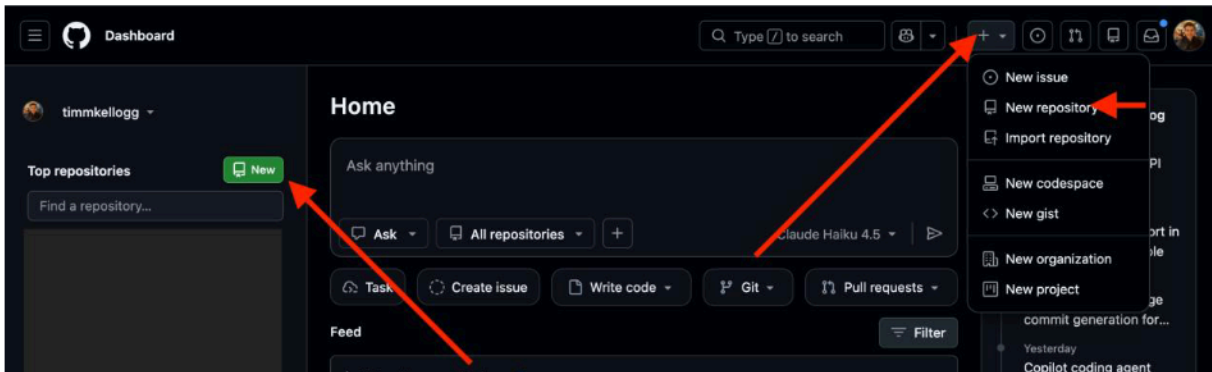
```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test
$ git init
Initialized empty Git repository in C:/Users/timmk/Documents/git-test/.git/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$
```

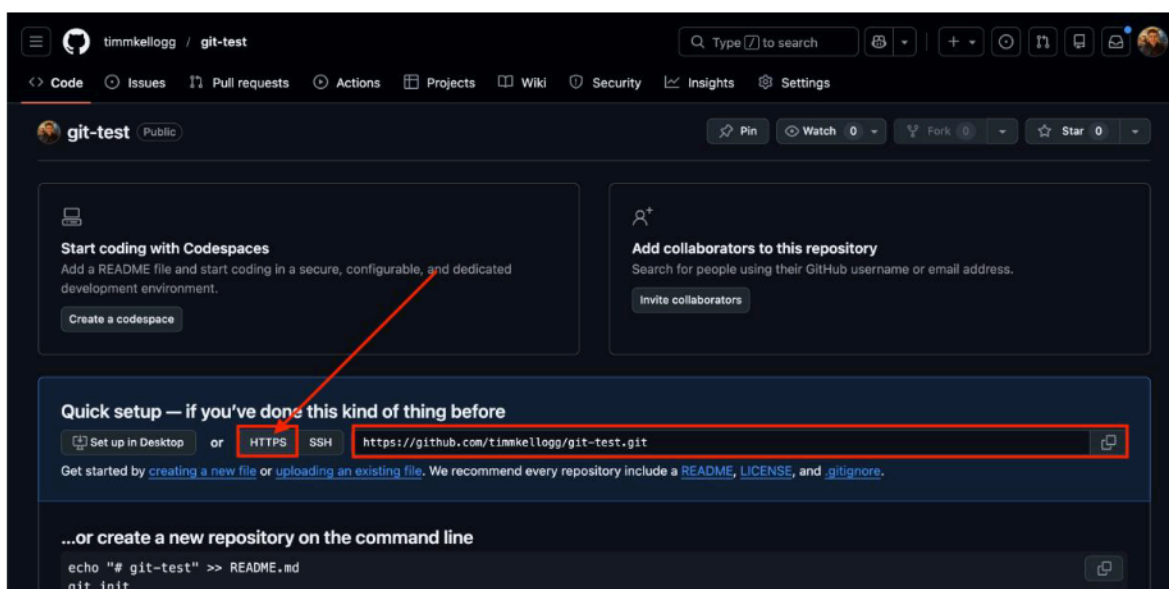
- This command:
 - ◇ Turns a normal folder into a Git repository
 - ◇ Creates a hidden Git metadata folder
 - ◇ Enables version tracking
- Your project is now version controlled locally

2.C.3 Creating a Remote Repository

- To create a remote repository on GitHub:
 1. Log in to [GitHub.com](https://github.com) (create an account if you don't have one yet)
 2. Click **New repository**



3. Enter a repository name
 4. Leave all other options default
 5. Click **Create repository**
- GitHub will generate a **repository URL** – you will need this later



2.C.4 Connecting Local to Remote

- After creating the repository on GitHub, connect it to your local repository
- Navigate to your local repository (`git-test`) folder in Git Bash, and run:

```
git remote add origin <your-repository-url-here>
```

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git remote add origin https://github.com//timmkellogg/git-test

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$
```

- This command:
 - ◇ Links your local repository to the remote
 - ◇ Tells Git where to push commits
- You now have:
 - ◇ A local repository
 - ◇ A remote repository
 - ◇ A connection between them

2.C.5 Adding and Committing Changes

- Inside your `git-test` folder, create a file:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ touch notes.txt
```

◇ Git now detects a change

- Stage the file using `git add <file-name>`:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git add notes.txt
```

◇ Staging selects what changes will be included in the next commit

- Create a commit using

`git commit -m "<helpful message here>":`

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git commit -m 'added notes.txt file'
[main (root-commit) a336942] added notes.txt file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 notes.txt
```

◇ You **must** provide a message with your commit

- A commit:

◇ Records a snapshot of your project

◇ Adds a descriptive message of the change

- Commits exist locally until pushed

2.C.6 Pushing to the Remote Repository

- Push your commit to GitHub using

```
git push -u origin main:
```

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 221 bytes | 221.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/timmkellogg/git-test
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- This command:

- ◇ Sends your local commits to the remote repository
- ◇ Establishes the upstream branch (don't worry about this right now)

- After the first push, future pushes only require `git push`:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ touch new-file.txt

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git add new-file.txt

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git commit -m 'added new-file.txt'
[main b378c22] added new-file.txt
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 new-file.txt

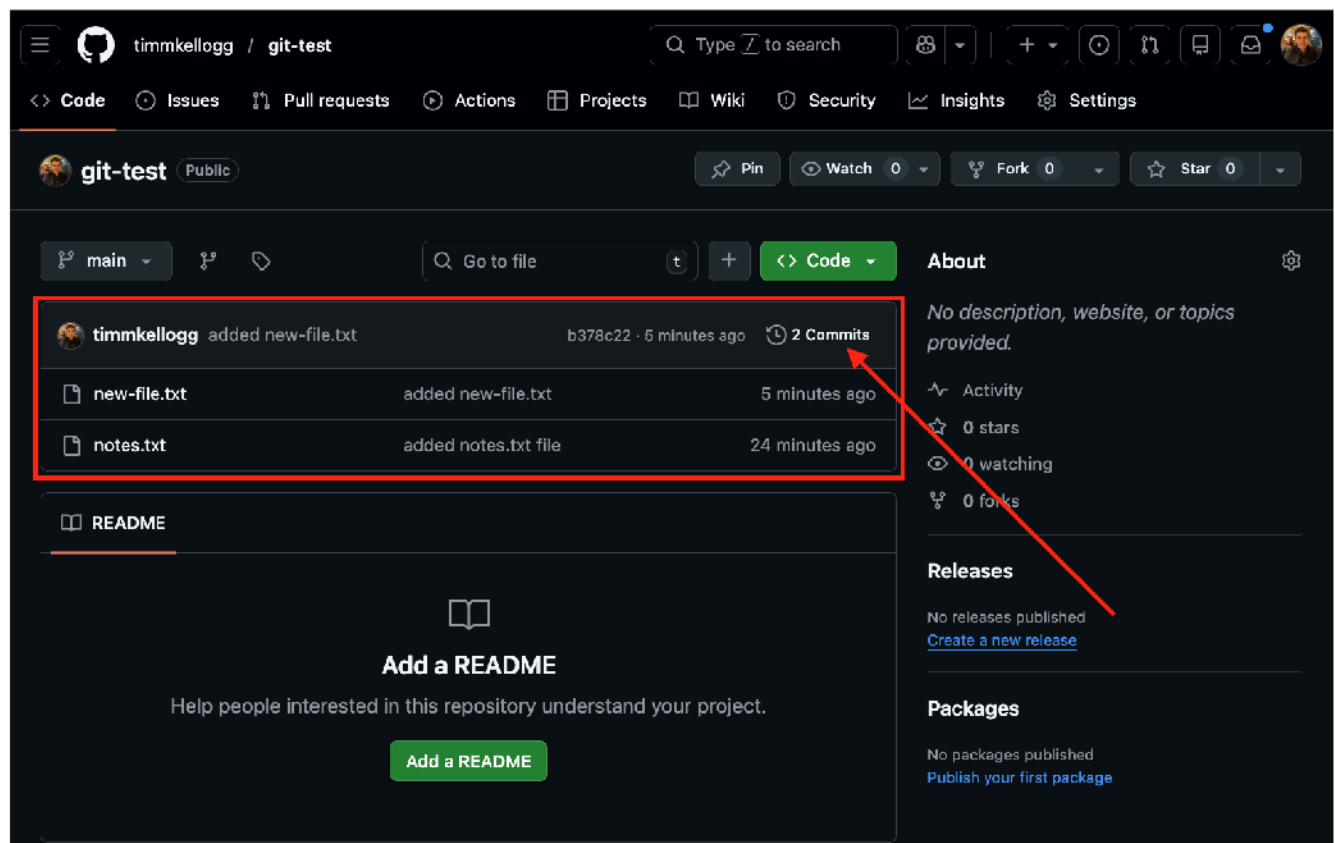
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$ git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 252 bytes | 252.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/timmkellogg/git-test
 a336942..b378c22  main -> main
```

2.C.7 Authenticating with GitHub

- When pushing for the first time, you will be prompted to authenticate your GitHub account
- GitHub no longer accepts account passwords for Git operations
- Instead, we will use a Personal Access Token
- Generate a token on GitHub:
 1. Go to **Settings**
 2. Select **Developer settings**
 3. Choose **Personal access tokens > Tokens (classic)**
 4. For the **Note**, type “Data Analytics Academy Git Access”
 5. Under **Select scopes**, Enable `repo` permissions
 6. Click **Generate token**
 7. Copy your token
- During `git push`:
 - ◇ Enter your GitHub username
 - ◇ Paste the token as the password
- Git will cache your credentials for next time

2.C.8 Verifying Your Work

- After pushing:
 - ◇ Visit your repository on GitHub
 - ◇ Confirm your file appears
 - ◇ Confirm your commit message is visible
 - ◇ Review the commit history



- If your changes do not appear on GitHub, confirm that you:
 - ◇ Staged the files (`git add <file-name>`)
 - ◇ Committed the changes (`git commit -m "<your message>"`)
 - ◇ Pushed to the remote (`git push -u origin main`)

2.C.9 Cloning a Repository

- Cloning creates a new local copy of an existing remote repository
- In Git Bash, run `git clone <repository-url>`:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents
$ git clone https://github.com/timmkellogg/git-test
Cloning into 'git-test'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents
$ cd git-test/

timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/git-test (main)
$
```

- This command:
 - ◇ Downloads the repository
 - ◇ Creates a new folder
 - ◇ Connects the local copy to the remote automatically
- Cloning is used when:
 - ◇ Starting work on an existing project
 - ◇ Working on a shared repository
 - ◇ Recovering a project on a new device
- Cloning differs from `git init`
 - ◇ `git init` creates a **new repository**
 - ◇ `git clone` creates a **copy of an existing one**

2.C.10 Git Beyond the Basics

- Git is powerful
- This module covers the foundation
- Git also supports:
 - ◇ Branching and merging changes
 - ◇ Resolving conflicts
 - ◇ Collaborative workflows
 - ◇ Advanced history management
 - ◇ And more
- You are not expected to master these features now
- If you can:
 - ◇ Initialize a repository
 - ◇ Connect it to a remote
 - ◇ Add and commit changes
 - ◇ Push to GitHub
 - ◇ Clone an existing repository
- You have the foundation
- Everything else builds from this workflow